

Hyperledger Fabric can be performed on Windows, but the recommended and easiest method is:

Recommended Setup on Windows

Use:

- Windows 10/11
- WSL2 (Windows Subsystem for Linux)
- Ubuntu inside WSL
- Docker Desktop

Because Hyperledger Fabric tools and scripts are Linux-based.

Best Practical Setup

Architecture on Windows

Windows

↓

WSL2 Ubuntu

↓

Docker Desktop

↓

Hyperledger Fabric Network

Step-by-Step Setup on Windows

Step 1 — Install Docker Desktop

Download:

[Docker Desktop](#)

During installation:

- Enable WSL2 integration

- Restart PC if required
-

Step 2 — Install WSL2

Open PowerShell as Administrator:

```
wsl --install
```

Restart system.

Step 3 — Install Ubuntu

From Microsoft Store install:

- Ubuntu

Open Ubuntu terminal.

Set:

- username
 - password
-

Step 4 — Verify Docker in WSL

Inside Ubuntu terminal:

```
docker --version
```

```
docker ps
```

If Docker works → setup is correct.

Step 5 — Install Required Packages

Inside Ubuntu:

```
sudo apt update
```

```
sudo apt install curl git nodejs npm -y
```

Check:

```
node -v
```

```
npm -v
```

Step 6 — Download Fabric Samples

```
curl -sSL https://bit.ly/2ysbOFE | bash -s
```

This downloads:

- Fabric binaries
- Docker images
- Sample chaincodes

Step 7 — Start Network

```
cd fabric-samples/test-network
```

```
./network.sh up
```

If successful, Docker containers start.

Check:

```
docker ps
```

You'll see:

- peer containers
- orderer
- ca containers

Step 8 — Create Channel

```
./network.sh createChannel
```

Step 9 — Deploy Chaincode

```
./network.sh deployCC \
```

```
-c channel1 \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccl javascript
```

Step 10 — Test Query

```
peer chaincode query \  
-C channel1 \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

Important Windows Notes

DO NOT use:

- Git Bash alone
- CMD
- PowerShell directly for Fabric

Because:

- Fabric scripts use Linux shell commands
- Docker networking becomes problematic

Use Ubuntu WSL terminal.

Minimum System Requirements

Requirement Recommended

RAM	8 GB+
CPU	i5/Ryzen 5

Requirement Recommended

Disk 20 GB free

Windows Windows 10/11

Common Errors on Windows

1. Docker daemon not running

Solution:

- Start Docker Desktop
-

2. Permission denied

Use:

```
chmod +x network.sh
```

3. WSL integration issue

Docker Desktop →

Settings →

Resources →

WSL Integration →

Enable Ubuntu

4. Port already in use

Stop old containers:

```
docker rm -f $(docker ps -aq)
```

Can Fabric Run Without WSL?

Technically possible using:

- VirtualBox

- VMware
- Linux VM

But WSL2 is:

- faster
 - lighter
 - officially preferred for Windows users
-

Recommended Practical Workflow for College

Use:

- VS Code
- WSL Ubuntu terminal
- Docker Desktop

VS Code extension:

- [Remote - WSL Extension](#)

This lets you code Fabric projects directly inside Ubuntu.

What to Write in Practical File

Aim

To implement and deploy chaincode using Hyperledger Fabric on Windows using WSL2.

Software Used

- Windows 11
- Docker Desktop
- WSL2 Ubuntu
- Node.js
- Hyperledger Fabric

Result

Successfully deployed and tested chaincode transactions in Hyperledger Fabric network.

What is Chaincode in Hyperledger Fabric?

Chaincode is the **smart contract** in Hyperledger Fabric.

It contains business logic such as:

- Create Asset
- Transfer Ownership
- Query Data
- Update Records

Fabric supports chaincode in:

- Go
- JavaScript (Node.js)
- Java

For learning, JavaScript chaincode is easiest.

Practical Goal

We will create a simple blockchain application:

Example:

Store student records on blockchain.

Operations:

1. Add Student
 2. Read Student
 3. Update Student
 4. Delete Student
-

Architecture

Client Application

↓

Hyperledger Fabric Network



Peers execute Chaincode



Ledger stores data

Software Requirements

Install these:

Software	Purpose
Docker	Containers
Docker Compose	Manage Fabric containers
Node.js	Chaincode
Git	Clone samples
cURL	Download binaries

Step 1 — Install Hyperledger Fabric Samples

Open terminal:

```
curl -sSL https://bit.ly/2ysbOFE | bash -s
```

This downloads:

- Fabric binaries
 - Docker images
 - Sample projects
-

Step 2 — Go to Test Network

```
cd fabric-samples/test-network
```

Step 3 — Start Fabric Network

```
./network.sh up
```

This creates:

- Orderer
- Peer nodes
- Certificate Authorities

Step 4 — Create a Channel

```
./network.sh createChannel
```

Channel = private communication layer.

Step 5 — Create Chaincode Folder

Go to:

```
cd ../asset-transfer-basic/chaincode-javascript
```

This already contains sample chaincode.

You can study and modify it.

Step 6 — Understand Chaincode Structure

Main file:

```
lib/assetTransfer.js
```

Basic structure:

```
'use strict';
```

```
const { Contract } = require('fabric-contract-api');
```

```
class StudentContract extends Contract {

  async createStudent(ctx, id, name, age) {

    const student = {

      id,

      name,

      age,

    };

    await ctx.stub.putState(

      id,

      Buffer.from(JSON.stringify(student))

    );

  }

  async readStudent(ctx, id) {

    const data = await ctx.stub.getState(id);

    if (!data || data.length === 0) {

      throw new Error(`Student ${id} not found`);

    }

    return data.toString();

  }

}
```

```
}
```

```
module.exports = StudentContract;
```

Important Functions

Function	Purpose
putState()	Store data
getState()	Read data
deleteState()	Delete data
getStateByRange()	Read all records

Step 7 — Deploy Chaincode

Return to test-network:

```
cd ../../test-network
```

Deploy:

```
./network.sh deployCC \  
-c channel1 \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccl javascript
```

Explanation:

Parameter Meaning

ccn	chaincode name
ccp	chaincode path

Parameter Meaning

ccl chaincode language

Step 8 — Invoke Transaction

Initialize ledger:

```
peer chaincode invoke \
```

```
-o localhost:7050 \
```

```
--ordererTLSHostnameOverride orderer.example.com \
```

```
--tls \
```

```
--cafile
```

```
"${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/  
msp/tlscacerts/tlsca.example.com-cert.pem" \
```

```
-C channel1 \
```

```
-n basic \
```

```
-c '{"function":"InitLedger","Args":[]}'
```

Step 9 — Query Blockchain Data

```
peer chaincode query \
```

```
-C channel1 \
```

```
-n basic \
```

```
-c '{"Args":["GetAllAssets"]}'
```

Output:

```
[  
  
  {  
  
    "ID":"asset1",  
  
    "Color":"blue"
```

```
}  
]
```

Step 10 — Add Custom Student Record

Example invoke:

```
peer chaincode invoke \  
-C channel1 \  
-n basic \  
-c '{"function":"createStudent","Args":["1","Sanket","21"]}'
```

Step 11 — Read Student Record

```
peer chaincode query \  
-C channel1 \  
-n basic \  
-c '{"function":"readStudent","Args":["1"]}'
```

Output:

```
{  
  "id":"1",  
  "name":"Sanket",  
  "age":"21"  
}
```

Chaincode Lifecycle (Important for Viva)

1. Package

Convert chaincode into package.

2. Install

Install on peers.

3. Approve

Organizations approve.

4. Commit

Chaincode becomes active.

5. Invoke

Execute transactions.

Key Hyperledger Fabric Concepts

Concept	Meaning
Peer	Maintains ledger
Orderer	Orders transactions
Channel	Private subnet
Ledger	Blockchain database
MSP	Identity management
Endorsement Policy	Transaction approval rules

Common Viva Questions

What is Chaincode?

Smart contract/business logic in Fabric.

Difference between Ethereum Smart Contract and Fabric Chaincode?

Ethereum	Fabric
Public blockchain	Permissioned blockchain

Ethereum	Fabric
Solidity	Go/JS/Java
Gas fees	No gas
Open network	Controlled access

Why Fabric uses Docker?

To isolate chaincode execution securely.

What is endorsement?

Peers validating transactions before commit.

Typical Lab Practical Flow

Aim

Implement and deploy chaincode in Hyperledger Fabric.

Procedure

1. Install Fabric
2. Start network
3. Create channel
4. Write chaincode
5. Deploy chaincode
6. Invoke transactions
7. Query ledger

Result

Successfully deployed and executed chaincode on Hyperledger Fabric blockchain network.

Mini Chaincode Example (Full CRUD)

```
async create(ctx, id, name) {
```

```
    const obj = { id, name };
```

```
    await ctx.stub.putState(
```

```
        id,
```

```
        Buffer.from(JSON.stringify(obj))
```

```
    );
```

```
}
```

```
async read(ctx, id) {
```

```
    const data = await ctx.stub.getState(id);
```

```
    return data.toString();
```

```
}
```

```
async update(ctx, id, name) {
```

```
    const obj = { id, name };
```

```
    await ctx.stub.putState(
```

```
        id,
```

```
        Buffer.from(JSON.stringify(obj))
```

```
    );
```

```
}
```

```
async delete(ctx, id) {  
  
    await ctx.stub.deleteState(id);  
}
```

Important Folder Structure

fabric-samples/

|

├─ test-network/

├─ asset-transfer-basic/

| └─ chaincode-javascript/

| └─ lib/

| └─ index.js

| └─ package.json

Best Way to Practice

Start with:

1. Asset Transfer
2. Student Record System
3. Voting System
4. Supply Chain Tracking
5. Certificate Verification

These are common college practicals and interview projects.